MAJOR PROJECT · MASTER DATA ANALYTICS

Nexus

A real-time chat platform that combines WhatsApp-style private chats with Discord-style communities

Project Documentation & Viva Notes

Spring Boot 3.5 · Java 17 | MongoDB | Redis | WebSocket (STOMP) | JWT + RBAC | Next.js 16 · React 19 | Three.js | Docker

PRESENTED BY
Razi Ahmed

ROLL NO.
57

DEPARTMENT
Master Data Analytics

GUIDED BY
Prof. Omkar Sherkhane

Key numbers to remember

- **10** MongoDB collections; most important index **{roomId: 1, createdAt: -1}** (latest 30 messages in one index walk)
- **3** roles per server: Owner > Admin > Member (RBAC enforced by `AccessGuard`)
- JWT valid **72 hours**, passwords hashed with **BCrypt**
- Rate limits: **8 messages / 5 s**, **10 logins / minute** (Redis)
- **28** automated tests, all passing; tests found and fixed **2** real race-condition bugs
- Idempotency proven: **10** parallel retries of one message → **1** stored document
- Deployed for free: Vercel + Render (Docker, Singapore) + MongoDB Atlas + Upstash Redis; backend uses about **162 MB of 512 MB**
- Free server sleeps after **15 min** idle → about **1 min** cold start

Contents

1. 1. Project overview
2. 2. Architecture and backend design
3. 3. Database: MongoDB
4. 4. Real-time messaging
5. 5. Security and RBAC
6. 6. Redis and error handling
7. 7. Backend API
8. 8. Frontend
9. 9. Testing and deployment
10. 10. Likely viva questions (with short answers)
11. 11. Live demo script: the app, MongoDB Compass and Redis side by side

Each chapter explains one part of the project in simple words: **what** it does, **why** it was built that way, and **where** the code is.

1. Project overview

What is Nexus?

Nexus is a **real-time chat application** that combines two kinds of apps students already use:

- **WhatsApp side**: private 1-to-1 chats and groups, read receipts, typing indicator, last seen.
- **Discord side**: servers (communities) with text channels, roles, invite links and an audit log.

Everything updates **live**: when someone sends a message, it appears on the other person's screen instantly, without refreshing.

The problem

1. Students switch between two apps (WhatsApp for friends, Discord for class communities): two accounts and scattered conversations.
2. Simple chat projects ignore real-world problems: - a message is **lost** if the receiver is offline; - a message is **duplicated** when the network retries; - anyone can read or post in any room (no permissions).

Objectives

1. One app for DMs, groups, servers and channels.
2. Real-time delivery with read ticks, typing and online status.
3. Secure login (JWT) and role-based permissions (RBAC).
4. No lost messages and no duplicate messages.
5. A clean, documented, tested and deployed system.

Features

Area	Features
Accounts	Register, login with username or email, logout (token really revoked), edit profile and avatar colour
People	Search users, friend requests, online / last seen
Chats (WhatsApp)	1-to-1 chats, groups, add/remove members, read ticks (blue when read), typing, unread badges
Servers (Discord)	Servers, channels, Owner/Admin/Member roles, kick, promote, invite links, Discover, audit log
Messages	Live delivery, history with infinite scroll, reply, react, edit, delete, pin, search, @mentions
UI	3D landing page (Three.js), dark futuristic theme, works on desktop and mobile

The whole system in one picture

```
Phone / Laptop browser
       |  HTTPS (REST)  +  WSS (WebSocket)
       v
Frontend: Next.js on Vercel
       |
       v
Backend: Spring Boot in Docker on Render
  JWT filter → Controllers → Services (+ AccessGuard) → Repositories
       |                                  |
       v                                  v
MongoDB Atlas (permanent data)       Upstash Redis (fast counters)
```

Technology stack

Layer	Technology	Why
Frontend	Next.js 16, React 19, TypeScript, Tailwind CSS 4	Typed components, fast routing, easy deploy
3D / animation	Three.js (React Three Fiber), Framer Motion	Futuristic landing page
Backend	Java 17, Spring Boot 3.5	Layered design, dependency injection, security
Database	MongoDB	Chat messages are naturally documents
Cache	Redis	Fast atomic counters with automatic expiry
Real-time	WebSocket + STOMP	Server can push messages instantly
Security	Spring Security, JWT, BCrypt	Stateless login for REST and WebSocket
Tests	JUnit 5, Mockito, MockMvc	28 automated tests
Hosting	Vercel, Render (Docker), MongoDB Atlas, Upstash	Free cloud deployment

Where is the code?

Folder	Contains
`ChatAppBackend/`	Spring Boot API (Java)
`chat-frontend/`	Next.js website
`docs/`	Detailed engineering documentation

Folder	Contains
`docker-compose.yml`	Starts MongoDB + Redis locally
`render.yaml`	Render deployment settings

2. Architecture and backend design

Layered architecture

A request passes through clear layers. Each layer has **one job**.

```
Request
  → RequestIdFilter          gives the request an id for the logs
  → JwtAuthenticationFilter  checks the login token
  → Controller               receives the request, validates the DTO
  → Service (+ AccessGuard)  business rules + permission checks
  → Repository               reads/writes MongoDB
  → MongoDB
Errors from any layer → RestResponseExceptionHandler → standard JSON error
```

Layer	Package	Job	Example
Controller	`controler/`	Receive REST / WebSocket calls, validate input with `@Valid`	`RoomControler`, `ChatController`
Service	`service/` + `service/impl/`	Business rules	`ChatServiceImpl.sendMessage`
AccessGuard	`security/AccessGuard`	"Is this user allowed?" in one place	`requireRoomMember`, `requireServerRole`
Repository	`repository/`	Database queries (Spring Data)	`MessageRepository`
Mapper	`mapper/`	Entity -> DTO, loads users in batches	`MessageMapper`
Entity	`entity/`	MongoDB documents	`Message`, `Room`, `User`
DTO	`dto/`	What goes in and out of the API	`SendMessageRequest`, `MessageResponse`

Why thin controllers? The same `ChatService.sendMessage` is used by both the REST endpoint and the WebSocket endpoint. If the rules lived in controllers they would be written twice.

Why DTOs? They decide exactly which fields the API accepts and returns. The password hash never leaves the server, and a client cannot set fields like `senderId` (the sender always comes from the token).

Two ways the frontend talks to the backend

Channel	Used for	Example
REST over HTTPS	Request / response	Login, load chat history, edit profile
WebSocket (STOMP)	Live push from the server	New message, typing, read ticks, online status

Sending a message uses the WebSocket first and falls back to REST if the socket is down.

Design principles applied

Principle	Where in the code	What it gives
Single Responsibility	`AccessGuard` only checks permissions; `MessageMapper` only maps	Easy to read and audit
Open/Closed	New error = one new `ErrorCode` constant	The error handler never changes
Liskov Substitution	All custom exceptions extend `ChatAppException`	One handler works for all
Interface Segregation	Small interfaces: `RateLimitService`, `PresenceService`	Classes depend only on what they use
Dependency Inversion	Services use `RealtimeService`, not the WebSocket class directly	Delivery can be swapped later
DRY	`Response.ok/error`, `AccessGuard`, `RedisSafe`, `ReusableForm`	Code written once
KISS / YAGNI	No microservices, no Kafka	Simple, enough for this scale
Dependency Injection	Constructor injection (`@RequiredArgsConstructor`)	Classes are easy to test with mocks

Example request flow: login

1. `POST /api/v1/auth/login` with username (or email) and password.
2. DTO validated (both fields required).
3. Rate limit checked: max 10 attempts per minute (Redis).
4. User found in MongoDB; password checked with BCrypt.
5. Server creates a JWT (user id, username, unique id, expiry 72 h).
6. Response: `{ token, expiresAt, user }`. The frontend saves the token and sends it on every request.

Wrong password or unknown user gives the **same** error (`AUTH_402`), so attackers cannot find out which usernames exist.

3. Database: MongoDB

Why MongoDB?

- Chat data is naturally a **document**: a message with its reactions and mentions inside it.
- Messages are mostly **added**, and almost always read **by chat, newest first**, which fits one compound index perfectly.
- The flexible schema let me add reactions, pins and mentions without migrations.
- Trade-off: no joins and limited transactions, so rules are enforced with **single-document writes** and **unique indexes**.

The 10 collections

Collection	Stores	Important fields
`users`	Accounts and profiles	username (unique), email (unique), passwordHash (BCrypt), lastSeenAt
`rooms`	Every conversation: DM, group or channel	type (DIRECT/GROUP/CHANNEL), serverId, directKey, lastMessage
`room_members`	Who is in which room	roomId, userId, role, **lastReadAt** (read ticks)
`messages`	Every message	roomId, senderId, content, reactions, replyToId, **clientMessageId**, createdAt
`servers`	Discord-style communities	name, ownerId, discoverable
`server_members`	Who is in which server and their role	serverId, userId, role (OWNER/ADMIN/MEMBER)
`invites`	Invite codes	code (unique), uses, maxUses, expiresAt
`friendships`	Friend requests	requesterId, addresseeId, pairKey (unique), status
`notifications`	Notification inbox	userId, type (MENTION, FRIEND_REQUEST ...), read
`audit_logs`	Admin actions in a server	serverId, actorId, action (MEMBER_KICKED ...)

The most important design decision

The original project stored **all messages inside the room document**. That was changed because: 1. A MongoDB document can be at most **16 MB**; a busy chat would break. 2. Every new message rewrote the whole room, so two people sending at once could **overwrite** each other. 3. Paging through an array inside one document is awkward.

Now every message is **its own document** in `messages`.

Embedded vs referenced (how I decided): embed when the data is small and always read together (reactions, mentions, the `lastMessage` preview); reference when it grows without limit or changes on its own (messages, members).

Indexes (why queries stay fast)

Collection	Index	Used for
messages	`{roomId: 1, createdAt: -1}`	Latest 30 messages of a chat, older pages
messages	`{roomId, senderId, clientMessageId}` unique (partial)	Blocking duplicate messages
room_members	`{roomId, userId}` unique + `userId`	Permission check; my chat list
server_members	`{serverId, userId}` unique + `userId`	Permission check; my servers
rooms	`directKey` unique (sparse), `serverId`	One DM per pair; a server's channels
users	`username` unique, `email` unique	Login, no duplicate accounts
friendships	`pairKey` unique, `requesterId`, `addresseeId`	One relation per pair; my friends
notifications	`{userId, createdAt: -1}`	Newest notifications first
audit_logs	`{serverId, createdAt: -1}`	Newest audit entries first
invites	`code` unique	Opening an invite link

Without an index MongoDB reads every document (a "collection scan"), so the app slows down as data grows. **With** `{roomId, createdAt}` it jumps straight to one chat's newest messages and reads only 30. You can prove this in MongoDB Compass: Explain Plan shows **IXSCAN**.

Pagination

Chat history uses **cursor pagination**:

```
GET /api/v1/rooms/{id}/messages?size=30                      → newest 30
GET /api/v1/rooms/{id}/messages?size=30&before=<createdAt>   → the 30 before that
```

Why not `page=5`? Offset paging gets slower the deeper you go, and new messages arriving while you scroll would shift the pages (duplicates or gaps). A cursor is stable and always fast.

Consistency (no transactions needed)

- The message insert is **one document**, so it is atomic.
- Rules that must never break are **unique indexes** (one DM per pair, one membership, one message per clientMessageId).

- Derived data (the chat preview, unread counts) may be briefly out of date and can always be recomputed. This is called **eventual consistency**.
- MongoDB transactions need a replica set and add delay to the busiest operation, so they are not used.

Sample message document

```
{
  "_id": "6abf85f6e34f128c8521a76c",
  "roomId": "6abf7e2e6b10a52fd984707d",
  "senderId": "6abf7e2a6b10a52fd984703c",
  "clientMessageId": "c-3f1d9a0e-8c2b",
  "content": "Got it! Looks amazing @razi",
  "type": "TEXT",
  "mentions": ["6abf7e2a6b10a52fd984703b"],
  "reactions": { "🔥": ["6abf7e2a6b10a52fd984703b"] },
  "edited": false, "deleted": false, "pinned": false,
  "createdAt": "2026-10-02T10:22:46.120Z"
}
```

4. Real-time messaging

Why WebSocket?

Chat needs the **server to push** data the moment it happens. With polling (asking every few seconds) the app is either slow or wastes requests. A WebSocket keeps **one open two-way connection**.

STOMP is a small protocol on top of WebSocket. It adds destinations ("channels"), subscriptions and private per-user queues, and Spring supports it out of the box.

Destination	Direction	Carries
`/app/rooms/{roomId}/send`	browser -> server	A new message
`/app/rooms/{roomId}/typing`	browser -> server	"I am typing"
`/topic/rooms/{roomId}`	server -> room members	New/edited/deleted messages, typing, read ticks
`/user/queue/events`	server -> one user	Unread counts, notifications, room changes
`/user/queue/errors`	server -> one user	Errors in the same JSON format as REST
`/topic/presence`	server -> everyone	Online / offline

Sending a message, step by step

1. **Send**: the bubble appears instantly with a clock icon and a unique **clientMessageId** (optimistic UI).
2. **Verify**: the WebSocket was authenticated with the JWT when it connected.
3. **Check** (`ChatServiceImpl.sendMessage`): - Is the sender a member of the room? If not: 403. - Was this clientMessageId already saved? If yes: return the saved one (**no duplicate**). - Rate limit: max 8 messages per 5 seconds. - A reply must point to a message in the same room.
4. **Save** in MongoDB. From now on the message is safe.
5. **Deliver**: pushed to everyone subscribed to `/topic/rooms/{roomId}` .
6. **Notify**: Redis unread +1 for every other member; @mentions create notifications.
7. The sender's bubble is replaced by the saved message and shows a tick. When the receiver opens the chat the ticks turn **blue**.

Save first, deliver second. Saving must succeed or the request fails. Delivery is best effort: if nobody is listening, the message is still in MongoDB.

What if ...?

Situation	What happens
Receiver is offline	Nothing is lost. The message is in MongoDB and the unread counter went up. When they come back they see the badge and the history.

Situation	What happens
WebSocket disconnects	The browser reconnects every 3 seconds and re-subscribes automatically. Meanwhile sending falls back to REST.
Message never confirmed	After 8 seconds the bubble shows "Not sent, retry". The retry uses the **same** clientMessageId.
MongoDB is down	The save fails before any delivery, so nobody sees a message that was never stored. The user gets 503 DB_503.
Same message sent twice (retry)	The unique index `{roomId, senderId, clientMessageId}` lets only one copy in; the other request gets the stored one back.
Server restarts	Everyone reconnects automatically; presence counters are reset.

No duplicate messages (idempotency)

Idempotency = doing something twice has the same effect as doing it once. - The client creates a `clientMessageId` (UUID) once per message and reuses it on every retry. - MongoDB has a **unique index** on `{roomId, senderId, clientMessageId}`. - If two copies race, the database accepts exactly one; the other gets `DuplicateKeyException` and returns the winner. - Proven by a test: **10 parallel retries -> 1 stored message**.

Read receipts, typing and presence

- **Read ticks:** opening a chat sets `room_members.lastReadAt = now` and broadcasts a READ event. A message is blue when every other member's `lastReadAt` is after it.
- **Typing:** sent at most every 2.5 seconds while typing; nothing is saved; hidden after 5 seconds.
- **Online status:** Redis counts open connections per user. Online when the count goes 0 -> 1, offline when it goes back to 0 (so 3 open tabs = still online after closing one). `lastSeenAt` is saved in MongoDB.

5. Security and RBAC

Passwords

Passwords are hashed with **BCrypt** (salted and deliberately slow). The database never contains real passwords, and the hash is never sent to the browser.

JWT login

JWT (JSON Web Token) = `header.payload.signature`. - Payload: user id, username, a unique token id (`jti`), expiry (72 hours). - Signature: made with a secret key (`JWT_SECRET`), so nobody can change the token without the server noticing. - A JWT is **signed, not encrypted**: anyone can read the payload, so it contains no secrets.

Flow: 1. Login succeeds -> server returns the token. 2. The browser sends `Authorization: Bearer <token>` with every request. 3. `JwtAuthenticationFilter` checks the signature, the expiry and the logout list. 4. Valid -> the user is known for this request. Invalid -> 401 with code `AUTH_401` or `AUTH_403` (expired).

Why JWT? The server stores no sessions, so any server instance can check a token, and the same token works for the WebSocket.

Logout: a JWT stays valid until it expires, so on logout its `jti` is saved in Redis (`auth:blacklist:<id>`) until that time. The filter rejects it from then on.

Authentication vs authorization

- **Authentication** = who are you? (login, JWT)
- **Authorization** = what are you allowed to do? (roles, membership)

WebSocket security

Browsers cannot add headers to the WebSocket handshake, so: - the token is sent in the STOMP **CONNECT** frame and checked by `WebSocketAuthInterceptor`; - every **SUBSCRIBE** to `/topic/rooms/{id}` is checked: only room members can listen.

RBAC: roles per server

Roles are stored in `server_members.role`, because a role belongs to the **relation** between a user and a server (Razi can be Owner of one server and Member of another). Ranks: **Owner (3) > Admin (2) > Member (1)**.

Action	Member	Admin	Owner
Chat, react, reply, create invite	Yes	Yes	Yes
Edit / delete own messages	Yes	Yes	Yes

Action	Member	Admin	Owner
Delete others' messages, pin in channels	No	Yes	Yes
Create / delete channels, edit server, see audit log	No	Yes	Yes
Kick someone	No	Members only	Members and Admins
Promote / demote (Member <-> Admin)	No	No	Yes
Delete server	No	No	Yes
Leave server	Yes	Yes	No (must delete)

Rule: you can only act on someone **below** your rank.

Groups: the creator is Owner, others are Members; if the Owner leaves, the oldest member becomes Owner. **DMs:** no roles.

Where it is enforced: `AccessGuard` in the backend (`requireServerRole(serverId, userId, ADMIN)`). The frontend only hides buttons; calling the API directly as a Member still returns **403 Forbidden**.

Demo data: in "Nexus HQ", `razi` = Owner, `aisha` = Admin, `kabir` and `zoya` = Members.

Other protections

Threat	Protection
Password guessing	Max 10 login attempts per minute
Finding valid usernames	Same error for wrong user and wrong password
Pretending to be someone else	Sender always taken from the token, never from the request
Spam	Max 8 messages per 5 seconds
Bad input	Validation on every DTO (lengths, formats)
Leaking internals	500/503 errors return generic messages; details only in logs
Secrets in logs	Logs show method, path, status, request id; never tokens or passwords
Other websites calling the API	CORS allows only the real frontend address

6. Redis and error handling

Why Redis?

MongoDB is the **permanent memory** (users, messages). Redis is the **short-term memory** for small numbers that change on almost every action. Redis keeps data in RAM, has **atomic** commands (`INCR` , `HINCRBY`) so two updates at the same time are never lost, and keys can **expire automatically** (TTL).

Redis key	Type	Used for	When it changes
`presence:connections`	hash: userId -> number of open connections	Online / offline	+1 on connect, -1 on disconnect
`unread:<userId>`	hash: roomId -> unread count	Unread badges	+1 for others when a message is sent; removed when the chat is opened
`rate:<action>:<user>:<window>`	number with expiry	Rate limiting	Counts messages (5 s window) and logins (60 s window)
`auth:blacklist:<tokenId>`	string with expiry	Logout	Saved on logout, deleted automatically when the token would expire

What if Redis goes down? Every Redis call goes through the `RedisSafe` class: - presence moves to memory; - unread counts are calculated from MongoDB (messages after `lastReadAt`); - rate limiting lets requests through; - Redis is skipped for 30 seconds so the app does not slow down retrying.

The chat keeps working. (Tested: Redis stopped, messages still sent.)

Error handling

All errors follow **one JSON format**:

```
{ "success": false, "message": "Room not found", "errorCode": "ROOM_404",
  "requestId": "9f1c2a7b3d4e5f60", "timestamp": "02-10-2026 15:52:46" }
```

How it works: 1. Services **throw** custom exceptions, for example `new ForbiddenException(ErrorCode.NOT_ROOM_MEMBER)` . 2. Every custom exception extends `ChatAppException` , which carries an `ErrorCode` . 3. The `ErrorCode` enum knows its HTTP status (`ROOM_404` -> 404). 4. One global handler (`RestResponseExceptionHandler` , a `@ControllerAdvice`) converts every exception into the JSON above. 5. WebSocket errors use the same format, sent to `/user/queue/errors` .

Exception classes (only 6, on purpose): `ResourceNotFoundException` , `DuplicateResourceException` , `UnauthorizedException` , `ForbiddenException` , `BadRequestException` , `RateLimitException` . The status lives in the enum, so a new error is one enum line.

Status	Example codes	Meaning
400	VAL_400, REQ_400	Invalid input (fields listed in `errorMeta`)
401	AUTH_401, AUTH_402, AUTH_403	Not logged in / wrong password / session expired
403	AUTH_404, ROOM_403, SRV_403	Logged in but not allowed
404	USER_404, ROOM_404, SRV_404	Not found
409	USER_409, USER_410, FRD_409	Duplicate (username taken ...)
429	RATE_429	Too many requests
503	DB_503	Database temporarily down: retry
500	SYS_500	Unexpected bug (details only in logs)

requestId: `RequestIdFilter` gives every request an id. It is in the `X-Request-Id` header, in every log line and in error responses. A user can quote it and the exact log lines can be found.

Frontend side: every failure becomes one `ApiError` type, and one function (`handleError`) shows a friendly message, puts field errors under the right input, and logs the user out on 401.

7. Backend API

Base URL: `/api/v1`. Live documentation: `/swagger-ui.html`. All endpoints except register, login, invite preview and health need `Authorization: Bearer <token>`.

Main endpoints

Method	Endpoint	Purpose
POST	`/auth/register`	Create an account
POST	`/auth/login`	Log in, get a JWT
POST	`/auth/logout`	Revoke the token
GET	`/users/me` / PUT `/users/me`	My profile / update it
GET	`/users/search?q=`	Find people
GET	`/rooms`	My chats (DMs + groups) with unread counts
POST	`/rooms/direct`	Open or create a DM
POST	`/rooms/group`	Create a group
GET	`/rooms/{id}/messages?before=&size=30`	Chat history (cursor pagination)
POST	`/rooms/{id}/messages`	Send a message (REST fallback)
POST	`/rooms/{id}/read`	Mark as read (blue ticks)
PUT / DELETE	`/messages/{id}`	Edit / delete a message
POST	`/messages/{id}/reactions`	Toggle a reaction
POST	`/messages/{id}/pin`	Toggle pin
GET / POST	`/servers`	My servers / create a server
GET	`/servers/{id}`	Server details: channels and members
POST	`/servers/{id}/channels`	Create a channel (Admin+)
DELETE	`/servers/{id}/members/{userId}`	Kick (Admin+, lower roles only)
PUT	`/servers/{id}/members/{userId}/role`	Change role (Owner)
POST	`/servers/{id}/invites`	Create an invite link
POST	`/invites/{code}/accept`	Join with an invite
GET / POST	`/friends` , `/friends/requests`	Friends and requests

Method	Endpoint	Purpose
GET	`/notifications`	Notification inbox
GET	`/health`	Health check (public)

Example: send a message

Request:

```
POST /api/v1/rooms/6abf.../messages
{ "content": "See you at 5 @kabir", "replyToId": null, "clientMessageId": "c-3f1d9a0e-8c2b"
}
```

Response (201):

```
{ "success": true, "message": "Message Sent",
  "data": { "id": "6abf85f6...", "content": "See you at 5 @kabir",
            "sender": { "displayName": "Razi Khan", "online": true },
            "clientMessageId": "c-3f1d9a0e-8c2b", "reactions": [], "createdAt": "2026-10-
02T10:22:46Z" } }
```

Sending the same `clientMessageId` again returns the **same** message id (no duplicate).

Example: validation error

```
POST /api/v1/auth/register  { "username": "a", "email": "bad", "password": "1",
"displayName": "" }
→ 400
{ "success": false, "errorCode": "VAL_400", "message": "Please fix the highlighted fields",
  "errorMeta": { "username": "Username must be 3-20 characters", "email": "Enter a valid
email",
                 "password": "Password must be at least 6 characters", "displayName":
"Display name is required" } }
```

API design rules followed

- Nouns for resources, sub-paths for actions (`/messages/{id}/pin`), version in the path (`/v1`).
- The acting user always comes from the JWT, never from the request body.
- The same response envelope for success and errors.
- Safe to repeat: join, open DM, accept invite, send with the same clientMessageId.

8. Frontend

Built with **Next.js 16 (App Router), React 19 and TypeScript**, styled with **Tailwind CSS 4**.

Pages

Route	Page
`/`	3D landing page
`/login` , `/register`	Login / sign up (3D side panel)
`/app`	Dashboard: unread count, online friends, servers, notifications
`/app/chats` , `/app/chats/{id}`	WhatsApp-style chat list and chat
`/app/servers` , `/app/servers/{id}/{channel}`	Discord-style servers and channels
`/app/friends` , `/app/discover` , `/app/notifications` , `/app/profile`	Friends, public servers, inbox, profile
`/invite/{code}`	Invite preview and join

How the frontend is organised

Folder	Contains
`app/`	Pages (routes)
`component/`	UI: chat window, sidebars, forms, 3D scenes, style enums
`context/`	Shared state: Auth, Socket, AppState, Server
`services/`	One class per backend area (`RoomService` , `ServerService` ...)
`lib/`	`ApiError` , `errorHandler` , helpers
`types/`	TypeScript copies of the backend DTOs

Contexts (shared state): - `AuthContext` : token + current user; logs out automatically when the token expires. - `SocketContext` : one STOMP WebSocket connection; re-subscribes after reconnect. - `AppStateContext` : chat list, servers, unread badges, who is online.

Reusable form: `ReusableForm` + `InputField` (Formik + Yup) build every form (login, register, create server ...). The same validation rules as the backend give instant feedback; the backend still checks again.

Chat window (`ChatPage`)

- Optimistic sending (bubble shows instantly) with retry on failure.
- Infinite scroll upward to load older messages.
- Bubbles (WhatsApp style) for DMs/groups, flat rows (Discord style) for channels.
- Reply, react, edit, delete, pin, copy; @mention suggestions; emoji picker.
- Read ticks: one tick = sent, blue double tick = read.

3D design

Three.js (via React Three Fiber) draws a glowing crystal with orbit rings, floating chat bubbles and stars on the landing and login pages. It is loaded only in the browser, uses fewer particles on phones, and if WebGL fails an error boundary shows a gradient instead. The chat screens use plain CSS so they stay fast.

Mobile layout

- Under 768px the Discord-style side rail becomes a **bottom tab bar** (Home, Chats, Servers, Alerts, Me).
- The tab bar hides inside an open chat, like WhatsApp.
- Tap a message to see its actions (phones have no hover).
- Inputs use 16px text so iPhones do not zoom in.

9. Testing and deployment

Testing

Run with `./mvnw test` in `ChatAppBackend`: **28 tests, all passing**.

Test class	Type	What it protects against
`AuthServiceImplTest` (6)	Unit (Mockito)	Plain-text passwords, duplicate accounts, revealing which usernames exist, brute force
`ChatServiceImplTest` (9)	Unit (Mockito)	Non-members posting, duplicate messages, spam, quoting another room's message, delivering a message that was never saved
`RestResponseExceptionHandlerTest` (6)	Web (MockMvc)	Wrong status codes, missing field errors, leaking internal details on 500/503
`JwtServiceTest` (4)	Unit	Forged tokens, expired tokens, broken tokens
`PresenceServiceConcurrencyTest` (1)	Concurrency	64 parallel connections must give exactly one online and one offline event
`ChatAppBackendApplicationTests` (2)	Integration (real MongoDB)	10 parallel retries of one message must store exactly 1 document

Unit test vs integration test: a unit test checks one class alone with fake (mocked) dependencies and runs in milliseconds. An integration test runs the real application with a real database to prove things a mock cannot, like the unique index really blocking duplicates.

Bugs the tests found (and I fixed): 1. **Presence race:** a reconnect happening during cleanup could be erased, so a user looked offline while online. Fixed with an atomic `computeIfPresent`. 2. **Rate limit vs retries:** 10 parallel retries of the same message used up the rate limit, so some got 429. Now a message counts only after a successful, non-duplicate save.

Deployment (free)

Part	Service	Notes
Frontend	Vercel	Built from `chat-frontend`, served over HTTPS from a CDN
Backend	Render (free web service)	Docker container, Singapore region
Database	MongoDB Atlas (M0 free)	Same 10 collections; viewable in MongoDB Compass
Cache	Upstash Redis (free)	Optional; app works without it

Docker: a two-stage `Dockerfile`. Stage 1 builds the jar with Maven; stage 2 runs it on a small Java 17 runtime as a **non-root** user, with memory limits for the 512 MB free tier. It runs in about **162 MB**.

Configuration: all secrets are **environment variables** (`MONGODB_URI` , `JWT_SECRET` , `CORS_ORIGINS` , `SPRING_DATA_REDIS_URL` , `SEED_PASSWORD`), never in GitHub.

Cold start: Render's free server sleeps after 15 minutes without traffic and needs about 1 minute to wake. The website pings `/api/v1/health` as soon as a page opens so it starts waking early. Open the site a few minutes before a demo.

How it would scale

Users	Change needed
100 (college)	Current design is enough
10,000	Always-on Redis, MongoDB replica set, monitoring
100,000	Several backend servers behind a load balancer; shared WebSocket broker (RabbitMQ or Redis pub/sub); only `RealtimeService` changes
1,000,000	Shard `messages` by roomId, separate WebSocket gateway, message queue (Kafka) for notifications and search, archive old messages

Future scope

File and image sharing, push notifications, end-to-end encryption, voice channels, refresh tokens, "delivered" (grey double tick) receipts.

10. Likely viva questions (with short answers)

How to answer: say **what** it is, **why** you chose it, **where** it is in the code, and one **trade-off**. Short, confident answers beat long ones.

About the project

Q1. What is your project? Nexus is a full-stack real-time chat application. It combines WhatsApp-style private chats and groups with Discord-style servers and channels. The backend is Spring Boot with MongoDB, Redis and WebSockets; the frontend is Next.js with a 3D landing page.

Q2. What problem does it solve? Students use two apps: WhatsApp for private chats and Discord for communities. Nexus puts both in one app. It also solves engineering problems that simple chat apps ignore: messages lost when a user is offline, and duplicate messages when the network retries.

Q3. What were your objectives? One app for DMs, groups, servers and channels; real-time delivery with read ticks, typing and presence; secure JWT login with role-based permissions; no lost or duplicate messages; and a documented, tested, deployed system.

Q4. List the main features. WhatsApp side: 1-to-1 chats, groups, read receipts, typing indicator, last seen, reply, reactions, edit, delete, pin, unread badges. Discord side: servers, channels, Owner/Admin/Member roles, invite links, public server discovery, audit log. Also friends, notifications, @mentions and message search.

Q5. What makes your project different from a normal chat project? Reliability and engineering depth: messages are saved before they are delivered, duplicate sends are blocked by a unique database index, Redis failures do not break the app, every error has a typed code, and there are 28 automated tests including concurrency tests.

Q6. Who are the users and what roles exist? Any registered user. Inside a server there are three roles: Owner, Admin and Member. Inside a group the creator is Owner. Roles are per server, not global.

Technology and architecture

Q7. Explain your architecture in 30 seconds. The browser talks to the backend in two ways: REST over HTTPS for request/response (login, history) and a WebSocket for live push (messages, typing, ticks). In Spring Boot a filter verifies the JWT, a controller validates the DTO, a service applies business rules and permission checks, a repository saves to MongoDB, and a realtime service pushes events. One global handler converts every error into a standard JSON response.

Q8. Why Spring Boot? It gives a production-ready web server, dependency injection, validation, security and WebSocket support out of the box. It also matched the original project, so I evolved it instead of rewriting.

Q9. Why Next.js and React? Component-based UI, TypeScript for type safety, file-based routing, and a production build that deploys easily on Vercel.

Q10. What are the layers of your backend? Controller (receives requests, validates DTOs), Service (business rules; AccessGuard checks permissions), Repository (Spring Data MongoDB queries), and the database. Controllers stay thin so the same service works for both REST and WebSocket.

Q11. Why do you have a service layer? Business rules must behave the same for REST and WebSocket. `ChatService.sendMessage` serves both. Services are also easy to unit-test because their dependencies can be mocked.

Q12. What are DTOs and why use them? Data Transfer Objects define exactly what goes in and out of the API. They prevent leaking fields like the password hash, stop clients from setting fields like senderId, carry validation rules, and let the database change without breaking clients.

Q13. Why both REST and WebSocket? REST suits request/response actions and is easy to document (Swagger) and retry. WebSocket suits server push. Sending a message uses WebSocket first and falls back to REST if the socket is down.

Q14. What is STOMP? A simple messaging protocol on top of WebSocket. It adds destinations (`/topic/rooms/{id}`), subscriptions and per-user queues (`/user/queue/events`), so I did not have to invent my own protocol. Spring supports it natively.

Q15. Where did you apply SOLID? Single responsibility: AccessGuard only checks permissions, MessageMapper only maps. Open/closed: a new error is one ErrorCode constant. Liskov: every ChatAppException subclass is handled through the base type. Interface segregation: small interfaces like RateLimitService. Dependency inversion: services depend on the RealtimeService interface, not on the WebSocket template.

Q16. What is dependency injection? Spring creates objects and passes their dependencies through the constructor (Lombok `@RequiredArgsConstructor` with final fields). Classes become testable, because tests pass mocks through the same constructor.

MongoDB and the data model

Q17. Why MongoDB? Chat data is document-shaped (a message with embedded reactions and mentions), mostly appended, and almost always read by one key (roomId) in time order, which fits a compound index perfectly. The flexible schema let me add features without migrations. Trade-off: no joins and limited transactions, so rules are enforced with single-document writes and unique indexes.

Q18. How many collections and what are they? Ten: users, rooms, room_members, messages, servers, server_members, invites, friendships, notifications, audit_logs.

Q19. Why did you separate users, rooms and messages? The original design stored all messages inside the room document. That hits MongoDB's 16 MB document limit, rewrites the whole room on every message (lost updates under concurrency) and makes pagination hard. A separate messages collection gives fast inserts and an indexed history query.

Q20. Embedded or referenced, how did you decide? Embed when data is small, bounded and always read together: reactions, mentions, the lastMessage preview. Reference when it grows without limit or changes independently: messages and members.

Q21. How are the latest messages loaded efficiently? With the compound index `{roomId: 1, createdAt: -1}`. MongoDB jumps to that room in the index, reads the newest 30 entries and stops: no full scan, no in-

memory sort, same speed for a small or a huge database.

Q22. Why are indexes required? Without an index MongoDB scans every document, so queries slow down as the whole app grows. Unique indexes also enforce rules: one DM room per pair, one membership per user per room, one message per clientMessageId.

Q23. Which pagination did you use? Cursor pagination: `?before=<createdAt of the oldest loaded message>&size=30`. Offset pagination (skip) gets slower with depth and shifts when new messages arrive, causing duplicates or gaps while scrolling.

Q24. Do you use MongoDB transactions? No. They need a replica set and add latency. The message insert is a single atomic write. Derived data like the chat preview and unread counts is eventually consistent and can be recomputed.

Q25. What is the 16 MB limit? The maximum size of one MongoDB document. Unlimited messages in one room document would eventually break, which is why messages have their own collection.

Q26. How do read receipts work in the database? Each room_members row has `lastReadAt`. Opening a chat sets it to now. A message shows blue ticks when every other member's lastReadAt is at or after the message time.

Real-time messaging

Q27. Why WebSocket instead of polling? Chat needs the server to push data instantly. Polling is either slow or wasteful. WebSocket keeps one open two-way connection, so messages, typing and ticks arrive immediately.

Q28. Explain the flow when a user sends a message. 1. The bubble appears instantly with a unique clientMessageId. 2. It goes over the WebSocket; the JWT was verified on connect. 3. The service checks: room member? duplicate? rate limit? 4. The message is saved in MongoDB. 5. It is pushed to everyone subscribed to `/topic/rooms/{id}`. 6. Redis increments other members' unread counts; @mentions create notifications.

Q29. What happens if the receiver is offline? Nothing is lost. The message is saved in MongoDB first and the unread counter goes up. When the user returns, the chat list shows the badge and opening the chat loads the history.

Q30. What happens if the WebSocket disconnects? The client reconnects every 3 seconds and re-subscribes automatically. While disconnected, sending falls back to REST. A pending message can be retried with the same clientMessageId, so it is never duplicated.

Q31. Are saving and delivering the same operation? No, on purpose. Saving must succeed or the request fails. Delivery is best effort, because offline users get the message from history.

Q32. How do you prevent duplicate messages? The client generates a clientMessageId (UUID) per message and reuses it on retry. MongoDB has a unique index on `{roomId, senderId, clientMessageId}`. A retry finds the stored message and returns it; if two copies race, the index lets exactly one win. A test with 10 parallel retries stores exactly one message.

Q33. What happens if MongoDB fails while sending? The save throws before any delivery, so nobody sees a message that was never stored. The user gets 503 DB_503 with a request id, and the bubble shows "Not sent, retry".

Q34. How does the typing indicator work? The composer sends a typing event at most every 2.5 seconds while typing. Nothing is saved. Receivers hide it after 5 seconds if no stop event arrives.

Q35. How does online/offline presence handle multiple tabs? Redis keeps a connection count per user. Online when it goes from 0 to 1, offline only when it returns to 0; then lastSeenAt is saved in MongoDB.

Q36. What is optimistic UI? Showing the message immediately, before the server confirms it, then replacing it with the saved version when the server echo arrives (matched by clientMessageId).

Security and RBAC

Q37. How are passwords stored? Never in plain text. They are hashed with BCrypt, which is salted and deliberately slow, so a leaked database does not reveal passwords.

Q38. How does JWT authentication work? After login the server creates a token: header, payload (user id, username, unique id, 72 h expiry) and a signature made with a secret key. The client sends it as `Authorization: Bearer` on every request. A filter verifies signature and expiry; if valid, the user is set for that request. No sessions are stored.

Q39. Is a JWT encrypted? No, only Base64-encoded and signed. Anyone can read the payload, so it contains no secrets. The signature proves the server issued it and nobody changed it.

Q40. How do you log out with a stateless token? On logout the token's unique id (jti) is saved in Redis until the token would expire. The filter rejects blacklisted tokens with AUTH_403. The list cleans itself.

Q41. Authentication vs authorization? Authentication: who you are (login, JWT). Authorization: what you may do (for example, only a server admin can create channels).

Q42. Did you implement RBAC? Explain it. Yes, role-based access control per server. Roles Owner, Admin and Member are stored in server_members. Members chat, react and invite; Admins also delete others' messages, create channels and kick Members; only the Owner changes roles or deletes the server. You can only act on someone below your rank. AccessGuard enforces this in the backend and returns 403 otherwise.

Q43. Why are roles in server_members and not in users? A role belongs to the relationship between a user and a server. Razi can be Owner of one server and Member of another.

Q44. How is the WebSocket secured? Browsers cannot add headers to the WebSocket handshake, so the token is sent in the STOMP CONNECT frame and checked by an interceptor. Every SUBSCRIBE to a room topic is checked against membership.

Q45. Why is CSRF protection disabled? CSRF abuses cookies that browsers send automatically. Nexus sends the token explicitly in a header, which another website cannot make the browser do. If tokens moved to cookies, CSRF protection would be needed again.

Q46. How do you avoid leaking sensitive data? DTOs never include the password hash; email is shown only on your own profile; 500/503 errors return generic messages; logs never contain tokens or passwords; login gives the same error for an unknown user and a wrong password.

Redis, error handling and concurrency

Q47. Why did you use Redis? For small numbers that change on almost every action: presence, unread counts, rate limits and the logout blacklist. Redis is in memory, very fast, has atomic commands (INCR, HINCRBY) and keys that expire automatically.

Q48. What happens if Redis goes down? RedisSafe catches the failure, logs it once and skips Redis for 30 seconds. Presence moves to memory, unread counts come from MongoDB, rate limiting lets requests through. The chat keeps working.

Q49. Which Redis keys exist? `presence:connections`, `unread:<userId>`, `rate:<action>:<user>:<window>`, `auth:blacklist:<tokenId>`.

Q50. How does your exception handling work? Services throw custom exceptions extending ChatAppException with an ErrorCode (for example ROOM_404) that knows its HTTP status. One global handler (`@ControllerAdvice`) turns every exception into the same JSON envelope with success, message, errorCode and requestId. WebSocket errors use the same envelope.

Q51. Why only about six exception classes? The HTTP status lives in the ErrorCode enum, so ROOM_404 and USER_404 share one class. A new error is one enum line.

Q52. What is the requestId? A correlation id given to every request. It is in the X-Request-Id header, every log line and error responses, so an error report can be matched to the exact server logs.

Q53. How does the system handle concurrent requests? Rules are enforced by atomic database and Redis operations, not Java locks: unique indexes and atomic Redis counters. This also works across multiple servers.

Q54. Two users open the same DM at the same time? Both insert a room with the same directKey. The unique index accepts one; the other catches DuplicateKeyException and loads the winner's room.

Q55. What is idempotency? Doing an operation twice has the same effect as once. Re-sending a message with the same clientMessageId, accepting an invite twice or opening the same DM twice never creates duplicates.

Q56. What is rate limiting? Limiting how often a user can act, to stop spam and brute force: 8 messages per 5 seconds and 10 logins per minute, counted in Redis. Excess requests get 429 RATE_429.

Testing, deployment and scaling

Q57. How did you test the project? 28 automated tests: Mockito unit tests (register, login, messages, permissions), MockMvc tests for every error status, JWT tests, a concurrency test with 64 parallel connections, and an integration test against real MongoDB.

Q58. Did testing find real bugs? Yes, two race conditions: a reconnect could be erased during presence cleanup (fixed with atomic computeIfPresent), and 10 parallel retries used up the rate limit (now counted only after a successful, non-duplicate save).

Q59. Unit test vs integration test? A unit test checks one class with mocked dependencies and is fast. An integration test runs real components together, here a real MongoDB, to prove things mocks cannot.

Q60. How is the project deployed? Frontend on Vercel, backend as a Docker container on Render's free tier (Singapore), database on MongoDB Atlas, Redis on Upstash. All secrets are environment variables, never in GitHub.

Q61. Why Docker? It packages Java and the app the same way everywhere. Multi-stage: Maven builds the jar, a small Java runtime runs it as a non-root user with memory limits for 512 MB. It uses about 162 MB.

Q62. Why does the first load take about a minute? Render's free tier sleeps after 15 minutes without traffic. The website pings the health endpoint as soon as a page opens, so the server starts waking early.

Q63. How would you scale to 1 million users? Several backend instances behind a load balancer (JWT is stateless), a shared WebSocket broker (RabbitMQ or Redis pub/sub), a MongoDB replica set with messages sharded by roomId, a message queue (Kafka) for notifications and search, and monitoring with tracing.

Q64. Why not microservices or Kafka now? They solve problems of very large teams and traffic that a college-scale app does not have, and add complexity and failure points. The code has seams (RealtimeService, UnreadService) where they can be added later.

Q65. What trade-offs did you make? JWT in localStorage (simple, sensitive to XSS), a soft message rate limit so retries never use up quota, eventually consistent previews and unread counts, and an in-memory WebSocket broker that supports one server instance.

Q66. What is the future scope? File and image sharing, push notifications, end-to-end encryption, voice channels, refresh tokens, and "delivered" receipts.

Rapid-fire revision

Question	One-line answer
Backend language and framework?	Java 17, Spring Boot 3.5
Database?	MongoDB (Atlas in the cloud), 10 collections
Real-time technology?	WebSocket with the STOMP protocol
Password hashing?	BCrypt
Token type and lifetime?	JWT signed with HS256, valid 72 hours
Most important index?	`{roomId: 1, createdAt: -1}` on messages
How are duplicates stopped?	clientMessageId + unique index
Pagination type?	Cursor (before = createdAt), 30 per page
Server roles?	Owner, Admin, Member (RBAC per server)
Where are permissions checked?	AccessGuard in the service layer
Error format?	One JSON envelope: success, message, errorCode, requestId
No permission?	403 Forbidden
Database down?	503 DB_503
Rate limit hit?	429 RATE_429
Redis down?	RedisSafe falls back, app keeps working

Question	One-line answer
Number of tests?	28, all passing
Frontend?	Next.js 16, React 19, TypeScript
3D library?	Three.js via React Three Fiber
Hosting?	Vercel, Render (Docker), MongoDB Atlas, Upstash
Saved before or after delivery?	Before: save first, then deliver

Two-minute demo to offer: log in as `razi` on the laptop and `aisha` on a phone, send a message, show typing and blue ticks, refresh MongoDB Compass to show the new document, then open a server to show roles and the member list.

11. Live demo script: the app, MongoDB Compass and Redis side by side

Follow these steps in order. For each one: **do this in the app**, **then show this in Compass / Redis**, **then say this**.

0. Setup (before the professor arrives)

Window	Open
Laptop browser	The Nexus website (Vercel link), logged in as **razi**
Professor's phone (or a private window)	The same link, log in as **aisha**
MongoDB Compass	Your **Atlas** connection → database `chatapp`
Upstash console	Your Redis database → **Data Browser** tab

Open the website 2–3 minutes early: the free server needs about a minute to wake up.

Compass basics you will use in every step: - Click a collection on the left → documents appear. - **Refresh** (circular arrow) after every action in the app; Compass does not update by itself. - **Sort** box: `{ createdAt: -1 }` puts the newest document first. - **Filter** box: for example `{ type: "GROUP" }`. - Ids: a document's own id is `_id: ObjectId("...")`. When one document points to another (like `roomId`, `userId`), it is stored as a **plain string**, so filter with quotes: `{ roomId: "6abf7e2e..." }`.

Opening line: *"The app has 10 collections. I will do actions in the app and show exactly which documents are created or changed."*

---

1. Users: register and login

Do: show the login page. Optionally register a new user live.

Show `users`: a document has `username`, `email`, `displayName`, `avatarColor`, `lastSeenAt`, and `passwordHash: "$2a$10$..."`.

Say: *"Passwords are never stored. This is a BCrypt hash. On login the server compares hashes and returns a JWT token."*

Redis: a `rate:login:<username>:<window>` key appears for 60 seconds. *"Max 10 login attempts per minute, to stop password guessing."*

---

2. Online status (Redis presence)

Do: razi is logged in on the laptop; aisha logs in on the phone.

Show Redis → `presence:connections` **:** a hash like `razi-id → 1`, `aisha-id → 1`. Open a second tab as razi → `2`. Close it → `1`.

Show in the app: the green online dot next to aisha. When aisha closes the app, the dot turns grey and "last seen" appears.

Then show `users` **→ aisha →** `lastSeenAt` got updated.

Say: *"Redis counts open connections per user. Online when it goes 0 → 1, offline only when it reaches 0, so several tabs don't flicker. When the user goes offline, lastSeenAt is saved permanently in MongoDB."*

---

3. Friends and notifications

Do (razi): Home → Friends → **Add friend** → type `zoya` (or the user you registered) → Send.

Show `friendships` **:** a new document:

```
requesterId: "<razi id>", addresseeId: "<zoya id>",
pairKey: "<smaller id>:<bigger id>", status: "PENDING"
```

Show `notifications` (sort `{ createdAt: -1 }`): `type: "FRIEND_REQUEST"`, `userId: <zoya id>`, `read: false`.

Do (log in as zoya, or have it open): the bell icon shows a badge and a pop-up appears instantly. Accept the request.

Refresh `friendships` **:** `status: "ACCEPTED"` and `respondedAt` filled. `notifications` **:** a new `FRIEND_ACCEPTED` for razi.

Say: *"pairKey is unique, so two people can only ever have one friendship document, even if both send requests at the same time. The notification is saved in MongoDB and also pushed live over the WebSocket."*

---

4. One-to-one chat (DM), and the ticks

Do (razi): Chats → open **Aisha Patel** (or start a new chat with the search icon).

Show `rooms` with filter `{ type: "DIRECT" }` : `directKey: "<id1>:<id2>"` (unique, so only one DM per pair), and `lastMessage { preview, senderName, sentAt }`.

Show `room_members` with filter `{ roomId: "<that room's _id as a string>" }` : **2 documents**, one for razi and one for aisha, each with `lastReadAt`.

Sending a message and the ticks

Bubble shows	Meaning	What happened in the database
Clock icon	Sending	Not in MongoDB yet
Single grey tick ✓	**Saved on the server**	Document inserted in `messages`

Bubble shows	Meaning	What happened in the database
Double grey ticks ✓✓ (groups only)	**Read by some** members	Some members' `lastReadAt` ≥ message time
Double blue ticks ✓✓	**Read by everyone**	Every other member's `lastReadAt` ≥ message time
"Not sent, retry"	Not confirmed in 8 s	Retry uses the same `clientMessageId`, so no duplicate

Do (razi): close the chat on aisha's phone first, then send **"Hello professor"**.

Show `messages` (sort `{ createdAt: -1 }`): the new document at the top: `roomId`, `senderId`, `content`, `clientMessageId`, `createdAt`. The bubble shows a **single grey tick**.

Show Redis → `unread:<aisha id>` **:** `<roomId> → 1`. Aisha's chat list shows a badge **1**.

Do (aisha): open the chat on the phone.

Refresh `room_members` **:** aisha's `lastReadAt` jumped to now. **Redis:** the `unread:<aisha id>` entry for that room is gone. **On razi's laptop:** the tick turned **double blue**, live.

Say: *"Ticks are not stored inside each message. Each member has a lastReadAt in room_members. A message is read when the other person's lastReadAt is later than the message's createdAt. Opening the chat updates lastReadAt and pushes a READ event, so the ticks turn blue instantly."*

Bonus, typing indicator: aisha types without sending → razi sees "Aisha is typing...". *"Typing is sent over the WebSocket only. Nothing is saved."*

Bonus, reply / react / edit / delete: react with 🔥 → `messages.reactions: { "🔥": ["<razi id>"] }`. Edit → `edited: true`. Delete → `deleted: true` with the content cleared (soft delete: the document stays so replies still work).

---

5. Group chat

Do (razi): Chats → **New group** icon → tick 2 friends → name it "Viva Demo" → Create.

Show `rooms` (filter `{ type: "GROUP" }`): the new room with `name: "Viva Demo"`, `createdBy`.

Show `room_members` (filter `{ roomId: "<group id>" }`): razi has `role: "OWNER"`, the others `role: "MEMBER"`.

Show `messages` **:** a `type: "SYSTEM"` message "Razi Khan created the group...".

Show `notifications` **:** an `ADDED_TO_GROUP` notification for each member.

Do: send a message with `@aisha` in it. **Show** `messages` **:** `mentions: ["<aisha id>"]`. `notifications` **:** a `MENTION` for aisha (she gets a live pop-up). **Ticks in a group:** grey ✓✓ when **some** members have read it, blue ✓✓ when **all** have.

Say: *"Groups use the same rooms, room_members and messages collections as DMs. Only type changes. The creator is OWNER and can add or remove members; if the owner leaves, the oldest member becomes owner."*

6. Communities: servers and channels (Discord side)

Do (razi): click **+** (or the Servers tab on a phone) → **Server** → name "Data Analytics Class" → Create.

One action, many collections. Refresh each:

Collection	What appears
`servers`	The server: `name`, `ownerId`, `discoverable`
`server_members`	razi with `role: "OWNER"`
`rooms` (filter `{ type: "CHANNEL" }`)	3 channels: `general`, `announcements`, `off-topic`, each with `serverId`
`room_members`	razi in each channel (for unread and read tracking)
`messages`	SYSTEM welcome message in #general
`audit_logs`	`action: "SERVER_CREATED"`

Do: server menu (top of the channel list) → **Invite people** → copy the link → open it on aisha's phone → **Accept invite**.

Collection	What changes
`invites`	The `code`, `expiresAt`, and **`uses`** goes up by 1
`server_members`	aisha added with `role: "MEMBER"`
`room_members`	aisha added to every channel
`messages`	SYSTEM "Aisha Patel joined the server. Say hi!"
`notifications`	`SERVER_JOINED` for razi (the owner)
`audit_logs`	`MEMBER_JOINED`

Roles (RBAC)

Do (razi, Owner): open the member list → click aisha → **Make admin**. **Show `server_members`:** aisha's `role` changed `MEMBER → ADMIN`. **`audit_logs`:** `ROLE_CHANGED`.

Do: log in as a plain **MEMBER** (for example kabir in Nexus HQ): there is no "Create channel" button. Even calling the API directly returns **403 Forbidden**.

Do (Owner or Admin): server menu → **Audit log** → the same entries appear in the app.

Say: *"Roles live in server_members because a role belongs to the user-and-server pair. Owner > Admin > Member, and you can only act on someone below you. AccessGuard checks this in the backend, so hiding a button is not the only protection. Every admin action is written to audit_logs."*

7. Redis summary (show the Upstash Data Browser)

Key	What you see	Changes when
`presence:connections`	userId → open connections	Login/logout, opening/closing tabs
`unread:<userId>`	roomId → unread count	+1 when someone else sends; removed when the chat is opened
`rate:message:<userId>:<window>`	a small number	Each message sent; disappears after 5 s
`rate:login:<name>:<window>`	a small number	Each login attempt; disappears after 60 s
`auth:blacklist:<tokenId>`	"1" with a countdown (TTL)	On logout; deletes itself when the token would expire

Say: *"MongoDB is permanent memory, Redis is fast short-term memory for counters that change on almost every action. Redis commands like HINCRBY are atomic, so two messages at the same moment never lose a count. If Redis goes down, the app falls back to MongoDB and keeps working."*

---

8. Finish with indexes (shows database understanding)

Compass → `messages` **→ Indexes tab:** `room_created {roomId: 1, createdAt: -1}` and `idempotency_key` (UNIQUE, PARTIAL).

Explain Plan tab: Filter `{ roomId: "<a room id>" }`, Sort `{ createdAt: -1 }`, Limit 30 → **Explain** → shows **IXSCAN**, and documents examined ≈ 30.

Say: *"Without this index MongoDB would read every message of every chat. With it, it jumps straight to this chat's newest 30 messages, the same speed for 100 or 10 million messages. The second unique index is why a retried message can never be stored twice."*

Quick map: action → collections

Action in the app	Collections written	Redis
Register	users	–
Login	–	rate:login
Open app (socket connects)	–	presence:connections +1
Close app	users.lastSeenAt	presence:connections −1
Friend request / accept	friendships, notifications	–

Action in the app	Collections written	Redis
Open a new DM	rooms (DIRECT), room_members ×2	–
Send message	messages, rooms.lastMessage, room_members (sender lastReadAt), notifications (if @mention)	unread +1 for others, rate:message
Open a chat	room_members.lastReadAt	unread entry removed
React / edit / delete / pin	messages (+ audit_logs for pin in a channel)	–
Create group	rooms (GROUP), room_members, messages (SYSTEM), notifications	–
Create server	servers, server_members, rooms (CHANNEL ×3), room_members, messages, audit_logs	–
Create / accept invite	invites, server_members, room_members, messages, notifications, audit_logs	–
Change role / kick	server_members (+ room_members on kick), audit_logs	–
Logout	–	auth:blacklist